講義 20：Control Break 群組小計（授課順序：接在講義 11 之後）

> 對應練習：ex20 | 答案程式：`ZR_TR20_CONTROL_BREAK`

本講重點

- 群組報表的標準需求：明細 → 各組小計 → 總計
- Control Break 陳述式：`AT NEW` / `AT END OF` / `AT FIRST` / `AT LAST`
- `SUM`：把整組數值欄自動加總進 work area
- 兩條鐵則：**先 SORT**、**群組欄位放結構前面**
- AT 區塊內欄位被遮蔽成 `*` 的規則

1. 需求長相

「航班營收報表，**依航空公司小計**，最後一行總計」——傳統紙本報表的招牌格式：

```
=== 航班營收（依公司小計） ===
AA American Airlines
   0017 2026/01/03    380    159,494.31
   0017 2026/01/31    372    156,136.13
   小計                752     315,630.44
AZ Alitalia
   ...
   小計                ...
================================
總計                5,204   2,381,904.99
```

土法煉鋼要自己記「上一筆的 carrid」逐筆比對；ABAP 直接內建了這個場景的語法——Control Break。

2. 語法：LOOP 裡的 AT 區塊

```
SORT gt_rev BY carrid connid fldate.          " 鐵則一：先依群組欄位排序！

LOOP AT gt_rev INTO gs_rev.

  AT FIRST.                                   " 整個 LOOP 的第一筆之前
    WRITE / '=== 航班營收（依公司小計） ==='.
  ENDAT.

  AT NEW carrid.                              " 每組的第一筆之前：組頭
    WRITE: / gs_rev-carrid, gs_rev-carrname.
  ENDAT.
```

```
  WRITE: /5 gs_rev-connid, gs_rev-fldate,      " 明細行（一般輸出）
            gs_rev-seatsocc, gs_rev-revenue.

  AT END OF carrid.                            " 每組的最後一筆之後：小計
    SUM.                                       " 數值欄加總進 gs_rev
    WRITE: /5 '小計', gs_rev-seatsocc, gs_rev-revenue.
    ULINE.
  ENDAT.

  AT LAST.                                     " 整個 LOOP 的最後一筆之後：總計
    SUM.
    WRITE: / '總計', gs_rev-seatsocc, gs_rev-revenue.
  ENDAT.

ENDLOOP.
```

陳述式	觸發時機	典型用途
`AT FIRST ... ENDAT`	迴圈第一筆之前，執行一次	報表標題
`AT NEW 欄位`	該欄位（含其**左邊所有欄位**）值改變的那一筆之前	組頭
`AT END OF 欄位`	該組最後一筆之後	**小計**
`AT LAST ... ENDAT`	迴圈最後一筆之後，執行一次	**總計**

3. SUM：整組自動加總

`SUM` 只能寫在 `AT ... ENDAT` 區塊內：把**目前群組**（AT FIRST/AT LAST 則是全部）所有列的**數值型欄位**（i、p、f）加總，結果放進 work area 的對應欄位。所以小計行直接 `WRITE gs_rev-revenue` 就是組合計——不用自己宣告累加變數。

注意：SUM 是「所有數值欄」一起總——結構裡若有不該加總的數值欄（如單價），小計行**不要去印它**（值是無意義的總和）。

`gs_rev` 平常放的是目前這一列，為什麼可以拿來放合計？ 因為進出 `AT ... ENDAT` 時系統會暫時改寫 work area，離開時再還原（官方文件 `ABAPAT_ITAB`、`ABAPSUM`）。下面是實跑結果（2026-09-30，驗證程式 `ZR_TR20_SUM_CHK`，AA 兩筆 `seatsocc` 10、20）：

時間點	`gs_rev` 的內容	實跑輸出
1. 進 `AT END OF carrid` 之前	目前這一列，也就是這組的最後一筆	`AA American 0064 20`
2. 一進入 `AT END OF carrid`	群組鍵 `carrid` 保留；右邊的字元類欄位填成 `*`；其他欄位清成初始值	`AA ****** ** 0`
3. 執行 `SUM` 之後	數值欄填入**這一整組**的加總	`AA ****** ** 30`

時間點	gs_rev 的內容	實跑輸出
4. 離開 ENDAT 之後	系統把目前這一列重新放回 gs_rev	AA American 0064 20

所以：

- 小計行 WRITE gs_rev-seatsocc 印的是合計 30，不是最後一筆的 20。
- SUM 不是把合計加到最後一筆上：第 2 步已經清成 0，第 3 步才填入全組合計。
- ENDAT 之後的程式碼再用 gs_rev，拿到的又是正常的明細資料，不會被合計污染。
- SUM 只能搭配 LOOP AT ... INTO gs_rev：用 LOOP AT ... ASSIGNING（講義 16）執行到 SUM 會 dump（執行期錯誤（Runtime Error） SUM_NO_ASSIGNING）。

4. 兩條鐵則與遮蔽規則

鐵則一：先 SORT

Control Break 只認「相鄰列」：沒依群組欄位排序，同公司資料分散各處，AT NEW/AT END OF 會觸發好幾次，小計整組錯——**而且不報錯**。SORT 的欄位順序要跟群組層次一致。

鐵則二：群組欄位放結構最前面

AT NEW f 的比較規則是「f **加上它左邊所有欄位**」有任一變動就觸發。所以結構欄位順序要照「群組層次」排：

```
TYPES: BEGIN OF ty_rev,
         carrid   TYPE sflight-carrid,     " 群組欄位放最前
         carrname TYPE scarr-carrname,     " 跟 carrid 一對一，緊跟其後
         connid   TYPE sflight-connid,     " 明細鍵
         fldate   TYPE sflight-fldate,
         seatsocc TYPE sflight-seatsocc,   " 數值欄（要小計的）
         revenue  TYPE p LENGTH 12 DECIMALS 2,
       END OF ty_rev.
```

把 carrid 放在中間，前面的欄位一變就誤觸發——群組欄位排最前是保命慣例。

遮蔽規則：AT 區塊內右邊欄位變 *

進入 AT NEW f / AT END OF f 區塊時，work area 中 f **右邊**的字元類欄位全部被遮成 *、數值欄清為初始值（明細值沒有意義了，系統防止誤用）。上例組頭想同時印 carrid 與 carrname，就要用 AT NEW carrname（它與 carrid 一對一，且依規則「carrname 及其左邊的 carrid」任一變動才觸發，效果等同 AT NEW carrid，但兩個欄位都看得到）：

```
AT NEW carrname.                    " 用右邊那個欄位當斷點，組頭才印得到兩欄
  WRITE: / gs_rev-carrid, gs_rev-carrname.
ENDAT.
```

第一次看到 ***** 出現在報表上，幾乎都是這個遮蔽規則造成的。

5. 適用邊界

- AT 區塊搭配 LOOP AT ... INTO 或 ASSIGNING 都可以用，但**要用 SUM 就一定要 INTO work area**（見第 3 節）；用 ASSIGNING 時進出 AT 區塊不會遮蔽或還原資料，那一列維持原樣。
- 迴圈若加了 WHERE 條件或中途 DELETE，群組判斷可能失真——要過濾就**先把資料整理成乾淨的內表（Internal Table）**再 LOOP。
- 只要「總計」不要明細時，別用 LOOP+AT LAST，直接 SELECT SUM(...) GROUP BY（資料庫端彙總）或 COLLECT 更省——資料庫端彙總的完整寫法見講義 20a。

5.1 新式寫法對照（7.40 之後）：LOOP AT ... GROUP BY（內表分組，Grouping）

AT NEW / AT END OF 依賴「先排序、群組欄位放最前面、右邊欄位被遮成 *」這些規則。7.40 之後可以改用 LOOP AT ... GROUP BY，直接指定「依什麼分組」：

```
LOOP AT gt_rev INTO DATA(gs_rev)
     GROUP BY ( carrid = gs_rev-carrid carrname = gs_rev-carrname )
     INTO DATA(gs_grp).

  WRITE: / gs_grp-carrid, gs_grp-carrname.              " 組頭：每組一次

  LOOP AT GROUP gs_grp INTO DATA(gs_mem).               " 這一組的每一筆
    WRITE: /5 gs_mem-connid, gs_mem-fldate, gs_mem-seatsocc, gs_mem-revenue.
  ENDLOOP.

  gv_seats = REDUCE i( INIT s = 0
                       FOR m IN GROUP gs_grp
                       NEXT s = s + m-seatsocc ).       " 小計自己算
  WRITE: /5 '小計', 20 gv_seats.
ENDLOOP.
```

語法拆解：

- GROUP BY (名稱 = 值 ...)：括號裡定義群組鍵（Group Key），值相同的列歸成一組；可以放多個欄位。
- INTO DATA(gs_grp)：gs_grp 代表「群組本身」，可以讀群組鍵 gs_grp-carrid、gs_grp-carrname。
- 外層 LOOP **每組只跑一次**；要讀組裡的每一筆，用內層 LOOP AT GROUP gs_grp。
- 沒有 SUM：小計用 REDUCE ... FOR m IN GROUP gs_grp 自己加，只加需要的欄位，不會像 SUM 把單價也加起來。累加的是金額時，INIT 要寫 INIT r TYPE 金額型別，寫 INIT r = 0 會讓累加變數變成整數，小數被截掉。

跟傳統 AT 區塊比：

	AT NEW / AT END OF	LOOP AT ... GROUP BY
要先排序	要，而且順序要對	不用，同一組的列會被收在一起

	AT NEW / AT END OF	LOOP AT ... GROUP BY
群組欄位位置	要放結構最前面	任何欄位都可以
組頭讀得到群組欄位	右邊欄位被遮成 *	群組鍵完整可讀
小計	SUM 自動加所有數值欄	自己用 REDUCE 加需要的欄位

練習 20 的新式寫法版本見答案程式 ZR_TR20_CONTROL_BREAK_NEW（382 行輸出跟原版逐行比對相同）。維護舊程式時 AT 區塊非常常見，兩種都要看得懂。

6. 常見錯誤與陷阱

症狀	原因
同一公司出現多次小計	沒先 SORT（或 SORT 欄位跟 AT 欄位不一致）
組頭/小計行印出 *****	遮蔽規則：AT 區塊內右邊字元欄被遮——改用右側欄位當 AT 斷點或別印
小計數字大得離譜	SUM 加總了不該總的數值欄（如單價）還把它印出來
AT NEW 太常觸發	群組欄位左邊還有會變動的欄位——群組欄位移到結構最前
ASSIGNING 迴圈執行到 SUM 就 dump（SUM_NO_ASSIGNING）	SUM 只支援 LOOP AT ... INTO work area；AT 區塊本身可以搭配 ASSIGNING

7. 課堂練習

完成 ex20：SFLIGHT + SCARR JOIN 出營收明細，依公司做組頭、小計，AT LAST 總計，並實驗「不 SORT 會怎樣」與遮蔽規則。